

Lecture Notes

CS241 - Operating Systems and Computer Networks

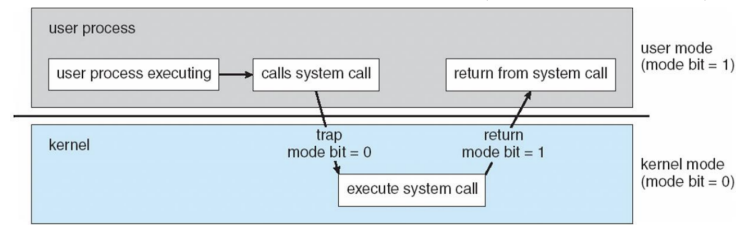
Intro

1. **Operating System (OS)** is a bridge between the user and the hardware managing program execution, resource allocation, I/O operations, accounting, file systems, communication, error detection, and security.
2. **Program Execution:** OS loads a program into memory at user request, executes, stops it, deallocating resources.
3. **I/O Operations:** Running programs may require I/O devices, managed by the OS through drivers and high-level interfaces for efficiency (no user coding needed) and safety (preventing simultaneous access to I/O).
4. **File system management** programs need to read/write files/directories, perform operations like **copy**, **move**, etc which may be subject to **permission management** (direct access prohibited). OS uses system calls to do these.
5. **Communication** between processes uses **shared memory** on the same computer and **message passing** on the network (e.g. browser-server requests).
6. **Error Handling** may occur in hardware: "cpu failed" or user programs: "segfault". OS either invokes error handling routine or shuts down the error-prone process.
7. **Resource Allocation** (memory, power) for multiple concurrent users/jobs is done through CPU **scheduling**, considering # of available processors, jobs and their speed.
8. **Accounting:** OS tracks resource usage of all running processes via **process control blocks (PCBs)** for administration or billing (e.g., cloud charging).
9. **Protection & Security:** OS ensures concurrent processes don't interfere with each other or the OS. **Encryption** and **authentication** prevent outside attacks.

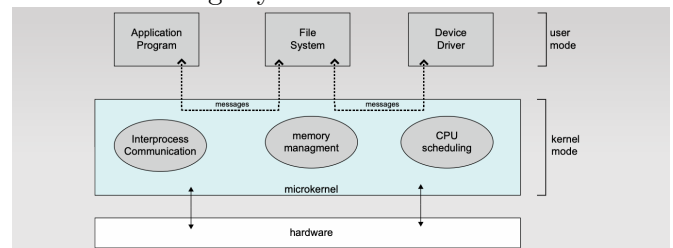
Kernel Management

1. **Kernel** (core of an OS) is loaded into the main memory at system startup (always running). Handles memory management, process scheduling, file handling etc.
2. **Kernel/User space** is part of the memory where kernel/user processes execute. Kernel space is protected from the user space, it can only be accessed via **system calls**.

3. **System calls** are invoked when a user process needs privileged operations so requires a service from the kernel.
4. **Dual mode operation:** hardware distinguishes OS (kernel mode, 0) and user (user mode, 1) operations. Privileged instructions only run in kernel mode, so system calls can temporarily switch between 0, 1 to do them. Unsupported in old hardware - security risk (MS-DOS crashes).



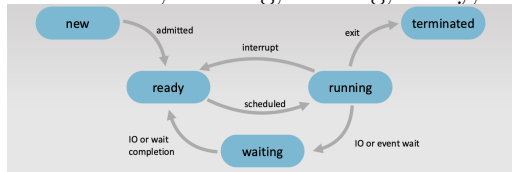
5. **Monolithic kernel** combines all functionality into one kernel; difficult to debug due to dependencies, but has little system call overhead.
6. **Layered Approach:** modular kernel design reduces dependencies between different parts ranging from bottom (hardware) to top layer (user). Layer k uses services of layer $k - 1$ and provides service to layer $k + 1$. Simple to construct, debug and use inter-layer interface, but difficult to define layers and has overhead in system calls that has to access multiple layers - recursive system call passing.
7. **Microkernel** only provides minimal process/memory management and inter-process communication - all non-essential components are implemented as system or user-level programs. Pros: easily extendable OS, secure and reliable since services run in user space. Cons: low performance due to large system call overhead.



8. **Loadable kernel modules:** kernel provides core services, others are dynamically loaded/unloaded at runtime.
9. **Hybrid Approach:** modern OS's mix the best traits of the approaches mentioned above (e.g. MacOS).

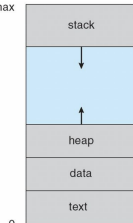
Processes 1

1. **Program**: passive entity stored on disk as an executable.
Process: active program loaded into memory (executing).
Process states: new, running, waiting, ready, terminated

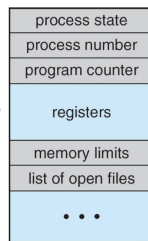


Virtual address space of a process has: max

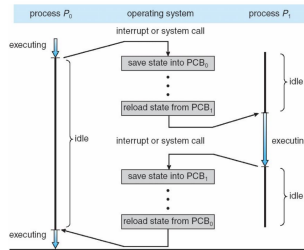
Text storing instructions, **Data** for global variables, **Heap** for dynamically allocated memory (malloc) and **Stack** storing local variables and function params. Space between stack and heap allows them to grow/shrink at program runtime.



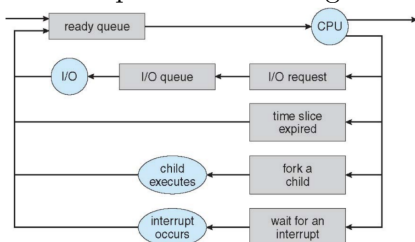
Process Control Block (PCB) has: process state, program counter (address of next instruct), CPU registers and scheduling info (priorities, scheduling, queue pointers), memory allocation, accounting info and I/O status. Helps execute process from where it left off.



Context Switch: OS saves current process state in PCB, loads next process's state, CPU resumes execution, but it's idle during the switch for a few ms: overhead, use OS schedulers.



5. **Short-term (cpu) scheduler** selects next process to execute from ready queue, invoked at least once every 100ms. **Long-term (job) scheduler** picks processes in new state to bring into main mem in ready queue, less frequent, minutes. Controls multiprogramming degree (# processes in memory), balances **process mix** of I/O and CPU-bound processes. Time-sharing systems (linux) don't have long-term scheduler, dump all processes to short-term.
6. **Concurrency** ability to run multiple processes simultaneously, quickly switching between two processes. **Job Queue** all new-state processes long term scheduler **Ready Queue**: all processes in ready state (short term), **Device Queue**: all processes waiting for I/O device.

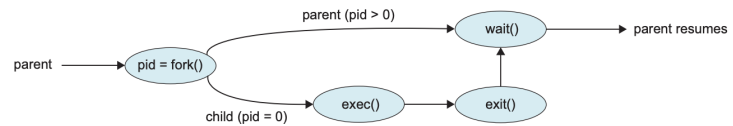


Processes 2

1. **OS process tree**: **parent process** creates **child process**, **siblings** if share a parent. All processes are uniquely identified by a positive integer **Process ID (PID)**.

2. **fork()**: system call for a process to create new process. Upon success, it creates a child process and returns its **real** positive PID (e.g. 100) to the parent and **fake** PID=0 to the child to signal "you're the copy, not the original". If it fails - return a negative value to the parent.

3. In **fork()** the child's address space is a copy of its parent's address space (heap, stack, data), text is usually shared. Since PID's are unique, the two are de-linked, so changes done by the child don't reflect in parent's address space. Each time **fork()** is called, number of processes is doubled.



4. **execvp()** system call erases old content of the address space, **including text** and replaces it with a new executable file (**executable prime**) and starts running it the newly loaded program. So, unless the call fails, the original code after the **execvp()** will never execute.

5. Process creation **options**: **Resource**: parent and children share either all, some or no resources; **Execution**: execute either concurrently or sequentially (parent waits for child to terminate); **Address space**: and copy parent's address space (**fork()**) or load a new program (**CreateProcess()**).

6. Process terminates either automatically after executing its last statement, or using **exit()** system call, which returns its **exit code** (status value) to parent, releases resources.

7. **Zombie Process**: child process terminated (resources released) but its exit status not collected by parent. Its entry remains in **process table** until parent receives exit status. The **wait()** system call collects exit statuses of children.

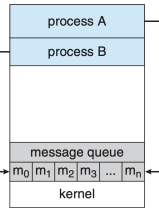
8. **Orphan Process**: parent exits without invoking **wait()**. **Init process** periodically **wait()**'s to collect exit status of all orphan processes and release their PID and process table entries.

9. Parent process may terminate a child process using **abort()** if child has exceeded allocated resources, the child task is no longer required or OS doesn't allow child to exist if parent terminated - so recursively closes all of the parent's children - **cascading termination**

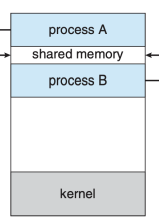
Processes 3

1. **Independent** process can't affect/be affected by other processes executing in the system, else it's **cooperating**.
2. **Interprocess communication (IPC)**: cooperating processes may need to communicate to: share data (shared file), speed up computation (multiprocessing), achieve modularity (multithreading) and convenience. Can use **Shared memory** or **Message passing** to communicate.

3. **Message Passing**: kernel provides a communication channel. Processes use system calls to exchange messages. Useful for small data (few conflicts) and is easy to implement on a distributed system, works better with more processing cores.



4. **Shared memory**: read/write from shared memory, kernel involved only in establishing the memory and permissions, everything else handled by processes. Typically faster - no system call overhead, but may suffer from cache coherency issues with more cores. Shared memory resides in the address space of one of the communicating processes, others need permission to access it. Once established, processes need to synchronise.

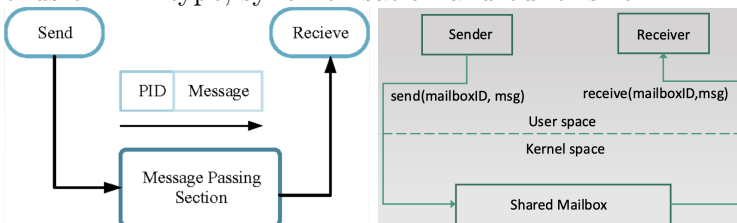


5. **Producer-Consumer** paradigm: process consumes data produced by another process (process can be both). Need shared memory to communicate - **shared buffer** can be filled by producer (wait if bounded & buffer full) and emptied by consumer (wait when buffer empty). When buffer is bounded, use a **circular queue** with in, out pointers.

```
buffer[in] = next_produced//producer: next free pos
buffer[out] = next_consumed//consumer: 1st full pos
in/out = (in/out + 1) % BUFFER_SIZE//move the ptrs
```

Buffer is *full* if $((in+1)\%BUFFER_SIZE)==out$ and *empty* if $in==out$, though also true if full, so has $\leq BUFFER_SIZE-1$ items. Instead, use counter, then *empty* if counter > 0 , now holds $\leq BUFFER_SIZE$ items.

6. **Message Passing System** has to have `send(m)` and `receive(m)` system calls for a message m - establishing a **communication link** (right fig.). Implementation depends on link type, synchronisation and buffer size.



7. **Direct communication**: processes must name each other explicitly (**symmetry**), or sender must name recipient (**asymmetry**): `send/receive(PID, m)`. Only 1 link established automatically between exactly 2 processes. **PID may change** - don't hardcode it in! (left fig.)

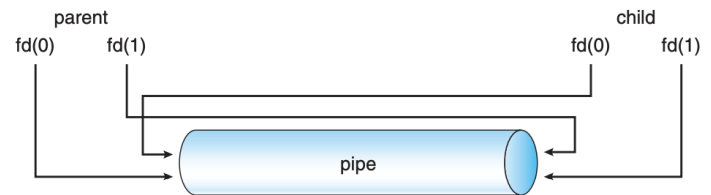
8. **Indirect Communication**: messages are passed through **ports** (mailboxes), use `send/receive(mailboxID, m)`. Link established only if processes share a common port. Can have multiple links per pair, each associated with (potentially) many processes, but only one port. When the owner process terminates, their mailbox disappears, consequent sender processes will be notified of this.

9. **Blocking send**: sender is blocked until their message is received by receiving process or the mailbox, **blocking receive**: receiver is blocked until the message is available. **Rendezvous**: both `send()`, `receive()` are blocking. Blocking is **synchronous**.

10. **Non-blocking send**: sending process sends the message and continues, **non-blocking receive**: receiver retrieves a valid or a null message. Can use non-blocking send with blocking receive and vice versa. Non-blocking is **asynchronous**.

11. Communication link is a buffer, can have **Zero capacity**: queue's max length is 0, so sender must block until recipient receives the message, **bounded capacity**: buffer has finite length, sender is blocked when full, and **unbounded capacity**: sender is never blocked.

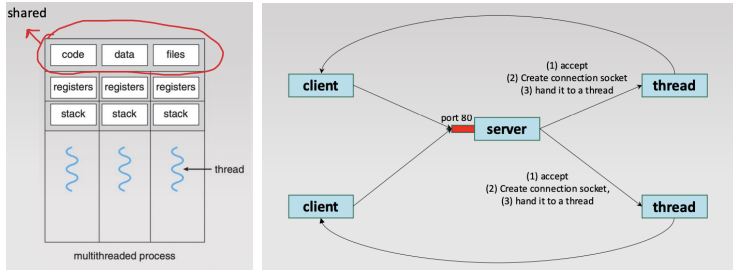
12. **Ordinary pipe** allows one-way communication through message passing. In Unix, pipe is a special type of file in-memory, not on disk, so only exists during runtime. The `pipe(fd)` system call returns an array of two file descriptors with data flowing from **write-end** `fd[1]` to **read-end** `fd[0]`. They can be accessed with `read()` and `write()` system calls and usually require parent-descendant relationship between the communicating processes: child process inherits open files from its parent, including its pipes. Can't reverse: two-way communication requires 2 pipes. Can't be accessed from outside the process that created it.



13. **Named pipes** exist outside of process runtime, so can be used at any time, more powerful. Bidirectional, no parent-child relationship required. Multiple processes can use it, usually has several writers. Create using `mkfifo()`.

Threads 1

1. **Thread** is a unit of CPU execution comprising thread id, program counter, register set and stack. Threads share code, data, heap, opened files, signals, etc (more than processes), so thread creation has less overhead than process creation. **Threading** is splitting a job into a number of separate threads, each of which can perform some task.

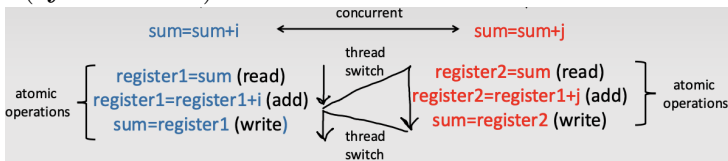


2. **Threads** allow for many concurrent tasks (*Scalability*), running cheaper than process creation, thread switching is faster than context switching (*Economy*). May allow continued execution when another thread is blocked (*Responsiveness*) and easier communication than shared memory/message passing (*Resource sharing*).
3. Identify independent tasks to run in parallel, balance their load (equal work) to justify parallelism, split data across processing cores, manage data dependencies (sync)
4. **Concurrency**: working on multiple tasks, **Parallelism**: doing so simultaneously. Concurr. need not mean Parall.
5. **Data parallelism** distributes subsets of the same data across multiple cores, each performing same operation. **Task parallelism** distributes different tasks across multiple cores. Usually mix both. **Amdahl's Law**:

$$\text{speedup} \leq \frac{1(\text{time before parallel.})}{S(\text{serial part}) + \frac{1-S}{N}(\text{parallelisable part})}$$

where S is the proportion of the task running sequentially (e.g. 0.5) and N is the parallelisation speed-up (e.g. 4x).

6. **Pthreads**: POSIX standard for thread creation. When multiple threads write to same location, need to ensure threads don't overwrite contents written by other threads (**synchronise**).



7. **Race condition**: threads race to become the last one to write on the shared memory, this should be avoided. **Mutex lock**: each thread must first acquire a lock to perform updates on shared variables. Only one thread can hold the lock at once.

Threads 2

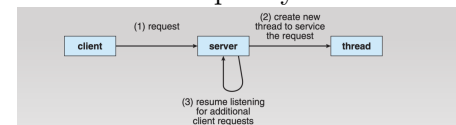
1. **Kernel-level threads**: managed by the OS, provide OS services (e.g. I/O) or execute user threads. **User-level threads**: exist in user processes, managed by thread libraries (kernel not involved). Must be associated with a kernel-level thread to execute on the CPU, otherwise invisible to the kernel.

2. **One-to-one** model: each user thread maps to a unique kernel thread. Pros: kernel is aware of all threads, so can reliably sync and schedule them in parallel, Cons: creating each user thread involves kernel, linking the two threads is expensive (system call overhead). Limited by OS thread management policies.

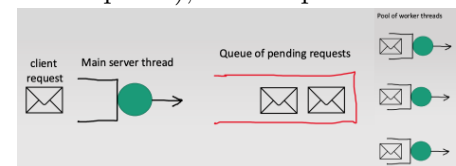
3. **Many-to-one** model: all user threads are mapped to a single kernel thread. Pros: less overhead and flexible thread management using user level library, Cons: multiple user threads can't run in parallel, one blocking causes the whole process to block.

4. **Many-to-many** model: user threads can be multiplexed onto $\leq$ number of kernel threads. Pros: parallelism in kernel threads, blocking call doesn't halt the process, configurable number of user and kernel threads: less overhead.

5. **One thread-per-request** strategy: separate thread for each request, have to create potentially too many threads, so high overhead and scales poorly under traffic.



6. **Thread Pool**: queue tasks for worker pool. Pros: faster to use existing thread than creating a new one, # threads bound to pool size (safe). Cons: potential wait if workers busy, deciding num workers (pool size) is hard, can use heuristics: num CPU's, memory size, expected num of concurrent client requests), so fixed pool reduces overhead.



7. **Signal**: used to notify a process that some event has occurred; generated by such occurrence, delivered to a process, then handled. **Synchronous signal**: internally generated by a process (e.g. illegal memory access) and delivered to the same process. **Asynchronous signal**: externally generated (e.g. Cntrl+C sends SIGINT termination signal), and usually send to another process (e.g. to terminate the program).

8. Multithreaded `fork()` duplicates all threads in a process unless followed by `exec()` (current thread replaces others), then only copy the calling thread.
9. **Condition variables** prevent worker threads from repeatedly checking the queue when there's no work to be done. A thread calls `wait(&cond_var, ..)` to pause/sleep until another thread wakes it up by signalling work availability using `signal(&cond_var)` (wake one thread) or `broadcast(&cond_var)` (wake all threads), then acquire a lock.
10. **Signal handlers** decide what the process does upon receiving a signal. **Default handler** is used by kernel (e.g. `SIGINT`), can be overridden by **user-defined handler**. Synchronous signals are delivered to thread generating the signal, asynchronous signals typically sent to all threads. Can deliver to specific thread with `pthread_kill(pthread_t tid, int signal)`.
11. **Thread cleanup**: main thread is waiting to join with the worker threads after they finish executing - collecting their results and terminating the process using `pthread_join()`. It then wakes up any sleeping threads with `broadcast()`, destroys the queue with `destroy_queue()` freeing all heap blocks, so no leaks possible.

Synchronisation

1. **Synchronisation** prevents race conditions when concurrent threads/processes access shared variables. **Critical Section (CS)** is part of the code updating such shared variables, process can have multiple CSs.
2. **Critical Section problem**: given a group of processes $\{P_0, P_1, \dots, P_N\}$, each of which has a CS, design a protocol (entry/exit code sections) s.t. no two processes can concurrently execute their CS. Threads share logical address space, processes may share memory, so their CSs may access the same shared resources.
3. **Solution** to critical section problem satisfy the following. **Mutual exclusion**: no other process allowed in another processes CS. **Progress**: if no processes executing in the CS, then any one of the waiting processes must be able to enter their CS. **Bounded waiting**: no one process should have to wait indefinitely to enter its CS while others are allowed to enter theirs repeatedly.
4. **Peterson's algorithm** solves the CS problem for two processes using two shared variables: `flag[2]` to indicate if the process is ready to enter their CS, and `turn` to manage who's turn it is to do so. It satisfies all three criteria above, but relies on busy waiting (repeatedly checking a

condition), which is inefficient. It may also fail on modern CPUs due to instruction reordering or caching, unless memory operations are strictly ordered.

Peterson's Algorithm

```
do { flag[i]=true; turn=j; // looking at Pi
  while (flag[1-i] and turn==1-i); // busy wait
  /*critical section*/ flag[i]=false /*remainder*/
} while(true);
```

Mutual exclusion: only one can hold `turn[i]` when `flag[0] = flag[1] = true`. *Progress*: No process in CS, then some P_k `flag[k]=true` and `turn=k` enters. *Bounded waiting*: process waits at most one turn since another resets `flag[j]=false` after exiting.

5. CS handling in OS: **preemptive kernel** allows process interruption, improving responsiveness, good for real-time systems, but more sync needed to manage race conditions. **Non-preemptive kernel** doesn't allow interrupts, so no race conditions (only one process active at a time).
6. **Locking** protects CS with locks that only one process at a time can have. `test_and_set(bool *target)` atomically returns `target`'s original value and sets it to "True". **Atomic lock** is non-interruptible, often implemented using hardware instructions.

Atomic hardware lock

```
bool test_and_set (bool *target) { // atomic lock
  bool val = *target; *target = True; return val; }
do { while (test_and_set(&lock)) {
  /*CS start*/ /*CS end*/ lock = false;
}} while (true);
```

but hardware usually inaccessible to programmers, infeasible in multiprocessor environment (message passing delays entry in CS, inefficient). Use **synchronisation primitives**: mutex locks, condition variables, semaphores.

7. **Mutual Exclusion (Mutex) lock** ensures mutual exclusion. Caller waits until the lock is available, acquires it atomically, executes CS, then releases it.

Mutex lock

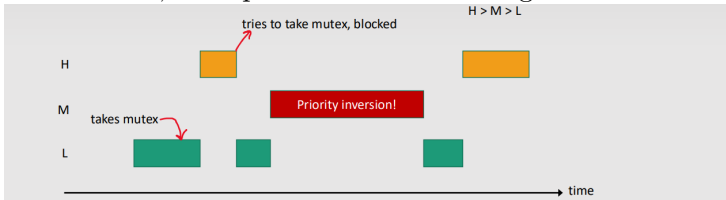
```
lock (&mutex); /*critical section*/ unlock(&mutex);
```

8. **Semaphore** can have integer value s , if $s > 0$: resource available, else: taken. Can only be accessed using atomic `wait()` (waits until $s > 0$, then $s--$) and `signal()`: $s++$. Both must be performed atomically.

Semaphore

```
void wait(int*s){while(*s<=0){/*busywait*/};*s--;}
void signal(int *s) { *s++ = 1; }
wait(&s); /*critical section*/ signal(&s);
```

9. **Deadlock**: multiple processes waiting indefinitely for an event that can only be caused by the waiting process (A waits for B, which waits for A).
10. **Starvation** (indefinite blocking): process waits indefinitely while others make progress (multiple processes waiting for semaphore, and signal call wakes up the same process repeatedly), fixed by randomly selecting process to wake up.
11. **Priority Inversion**: scheduling problem when lower-priority process holds a lock needed by higher-priority process. Solve using **priority-inheritance protocol**: processes accessing resources needed by higher-priority process inherit the higher priority until they're finished with the resource, then priorities revert to original.



12. **Bounded Buffer Problem**: n buffers, each can hold one item. Producer/consumer produces/consumes an item to/from the buffer, shouldn't produce/consume when all n buffers are full/empty.
13. **Readers and Writers Problem**: data set is shared among concurrent processes: readers (r) and writers (r,w). Allow multiple readers or a single writer at once - readers, writers can't access simultaneously. Readers are given preference when no process is active, so writers may starve.
14. **Dining Philosophers Problem**: n philosophers (*processes*) alternate thinking (*busy waiting*) and eating (*executing*) using n chopsticks (*semaphores*) from a shared bowl of rice (CS). Need to pick up 2 (left, right) chopsticks to eat, so neighbours can't eat at once. Possible deadlock: n philosophers pick up up left chopstick. Solution: either allow $\leq n - 1$ philosophers at once, only allow to pick up chopsticks if both are available, or use **asymmetric** solution: e.g. odd-num philosopher picks up left then right, even-num - vice versa.

Scheduling

1. Goal of CPU scheduling: increase CPU utilisation (have some process running at all times); basically a loop of alternating CPU and IO bursts - minimise idle time in both.
2. **Ready queue (RQ)** isn't necessarily FIFO, can be a PQ/linked list based on scheduling algorithm. Short-term (CPU) scheduler schedules processes from ready queue to be executed on CPU.

3. Performance measures: CPU **utilisation**: % of time CPU remains busy when there are jobs in RQ. **Throughput**: num of processes completed per time unit. **Turnaround**: time to complete a process. **Waiting**: time spent waiting in RQ. **Response**: time from request submission until first response, is a bit less relevant.
4. Types of scheduling: **Pre-emptive**: process execution is interrupted during CPU burst for scheduling (faster and more responsive but may introduce race condition); **Non-pre-emptive** - process sticks to the burst, no interrupts.
5. **Pre-emptive** scheduling can cause race cond, may leave data in inconsistent state if interrupted while updating shared data. Readers may see inconsistent data if access at time of writing, use critical sections and sync primitives.
6. **First Come First Served (FCFS) Scheduler**: processes assigned to CPU in order of their arrival - non-pre-emptive, performance depends on arrival sequence (if shorter jobs arrive first, then lower average waiting time). **Convoy effect**: long processes monopolise CPU, causing I/O-bound processes to wait despite being ready.
7. **Shortest Job First (SJF)**: schedule shortest jobs first, if a tie, use FCFS. It's optimal, gives minimum (**predicted**, because times potentially unknown) average waiting time. When new shorter process arrives during execution, pre-emptive SJF (optimal) switches to it, non-preemptive doesn't (finishes curr job). Special case of priority scheduling.
8. **Priority Scheduling**: numerical priorities associated with each process (CPU burst times in SJF), CPU allocated to highest priority process. Save low priority process from **starvation** using **aging** - gradually increase priority of waiting scheduled processes, usually +1 per time unit.
9. **Exponential Moving Average (EMA)**: len of n^{th} CPU burst t_n (most recent info), its predicted value τ_{n+1} , past history τ_n and its weight $0 \leq \alpha \leq 1$ ($\alpha = 0$ means that past history has no effect on the prediction), older terms matter less than recent. EMA : $\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$.
10. **Round Robin (RR) scheduling**: each process gets a **time quantum** q (small unit of CPU time $\simeq 10-100ms$), after which it's pre-empted and added to the end of RQ. Scheduler generally visits processes in order of arrival.

Each of N processes in RQ gets $1/N$ of CPU time in chunks of up to q time units at once. No process waits more than $(N - 1) \cdot q$ time units for its next turn.

RR is preemptive: timer interrupts every quantum to schedule next process, slower than SJF, but more responsive. Large q : RR same as FCFS, small q : too many context switches, overhead. Have to find optimal q .

Deadlocks

- System model:** system has finite resource types $R_1, \dots, R_m$ (locks, CPUs, I/O devices etc) each with W_i instances (3 printers) and set of processes $P = \{P_0, \dots, P_n\}$ - each utilising resources through **request-use-release**.
 - Set of processes P is in a **deadlock** when $\forall P_i \in P : P_i$ is waiting for an event that can only be caused by another $P_j \neq i \in P$, so the events (acquisition/release of some resources R) can't occur without intervention.
 - Conditions** for deadlock: **mutual exclusion:** only one process can use a resource instance at a time, **hold and wait:** $\exists$ process holding some resources while waiting for additional resources held by others; **no preemption:** resource can only be released voluntarily by the process after completing the task. **Circular wait:** there must be $\exists \{\tilde{P}_0, \dots, \tilde{P}_m\} \subseteq \{P_0, \dots, P_n\}$ s.t. $\tilde{P}_i$ waiting for $\tilde{P}_{(i+1) \bmod m}$. These conditions are necessary but not sufficient!
 - Resource allocation graph (RAG)** digraph $G=(V, E)$, V comprises processes P ($\circ$), and resource types R ($\square$). *Request edge:* $P_i \rightarrow R_j$ process P_i requested resource R_j , *Assignment edge:* $R_j \rightarrow P_i$ resource R_j was assigned to process P_i . No cycles in RAG implies no deadlock, cycle of single-instance resources and no free processes imply deadlock, otherwise cycles don't guarantee deadlock.
- Resource Type with 4 instances 

• P_i requests instance of R_j 

• Process 

• P_i is holding an instance of R_j 
- Deadlock prevention** by denying one of the necessary conditions: $\neg$ *mutual exclusion*: can't be prevented for non-shareable resources, $\neg$ *hold and wait*: ensure process gets all or none of its required resources, $\neg$ *no preemption*: if P_i requests unavailable resources, release the ones it holds and enqueue to be used later. $\neg$ *Circular wait*: ensure P_i holding R_n can't request $R_{m < n}$, only request later resources. Can be too restrictive, blocking harmless requests.
 - Deadlock avoidance:** less restrictive, dynamically grants request if resulting allocation leaves the system in a **safe state** (one where deadlock can't occur). Safe state implies no deadlock, but unsafe state is necessary but not sufficient for a deadlock.
 - Deadlock avoidance algorithm:** $\forall P_i$ declares max W_i of each R_i it may need. If granting R_i *immediately* leaves system in safe state - do so, else wait until can do.
 - Resource request algorithm:** determines if request can be granted immediately. Pretend request is granted and determine resulting state using Banker's, if so, grant immediately, else keep pending until state change.

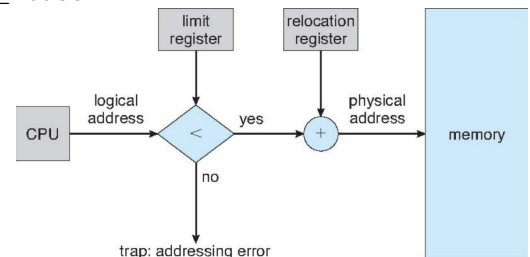
- Banker's safety algorithm** determines if set of processes P can execute safely by simulating each process P_i sequentially. If $\text{available}_j \geq \text{need}_j(\text{request})$: pretend that P_i , finished executing, so release resources ($\text{available}_j += \text{allocation}_j$). If all P_i complete in some order (**safe sequence**), the system is in a **safe state**, so no deadlock, however unsafe state doesn't guarantee deadlock. Need prior knowledge of each process's maximum needs, always allocates them to handle worst-case scenario (**max-allocation**).

	Allocation			Currently Available			Request		
	R_1	R_2	R_3	R_1	R_2	R_3	R_1	R_2	R_3
P1	0	0	0	1	2	1	1	0	0
P2	0	0	0				0	0	1
P3	0	0	0				0	0	0

REVISE AGAIN, page 331-337: available, max, allocation, need

Main Memory 1

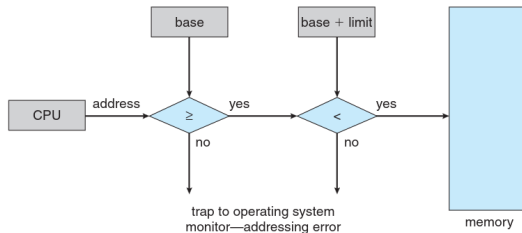
- Programs must be loaded into memory to execute, storing instructions and data at unique OS-allocated physical addresses - preventing illegal access or modification.
- Logical (Virtual) address** is generated by CPU to fetch instructions or read/write data. Differs from actual physical address at runtime, so needs to be translated to physical address for memory access/execution.
- Memory management unit (MMU):** hardware that translates runtime logical to physical addresses. In CMA, comprises **relocation register (base)** storing starting physical address of process's memory block, and **limit register (limit or boundary check or the size/range of its memory block)**. Only OS can modify these during process management (e.g. context switch). OS memory address usually $\leq$ base.



- Physical (Actual) address**, visible to MMU, is the actual memory location. The programmer is free from managing how the process is laid out in the memory, since its translation is handled by MMU.

5. **Contiguous Memory Allocation (CMA)**: each process occupies a fixed memory block (base to base+limit). Fixed number of partitions limits number of concurrent processes in memory, so new versions use variable partition sizes. OS has to decide base, limit for each process, overhead. Here, virtual memory is just an offset from base address.

6. **Memory protection** restricts processes to their own memory preventing errors and security risks. For CMA, the CPU checks user-mode generated logical addresses: $\text{base} \leq \text{base} + \text{logical}_i < \text{base} + \text{limit}$, otherwise - fatal error (attempted access to OS or other users' memory).



7. **Hole** is any available block of memory. OS keeps track of holes - dynamically allocating large enough holes to new processes. Process exiting creates a hole, adjacent holes are combined into single hole.

8. **Dynamic Storage allocation problem**: OS keeps track of holes to allocate to incoming processes. **First-fit**: first hole that's big enough. **Best-fit**: smallest hole that's big enough, searches entire list unless ordered (slow), produces smallest leftover hole (memory efficient). **Worst-fit**: largest hole, also search entire list, produces largest leftover hole that could be returned to the memory, but is slower and less memory-efficient than first and best fit, who are similar in performance (first fit slightly faster).

9. **Variable partition**: OS keeps a table indicating which parts of memory are available (holes) or occupied.

10. **External fragmentation**: enough memory to satisfy a request but non-contiguous fragments (first, best fit, variable partitioning). Keeps track of many small holes, high overhead **50% rule**: given N allocated blocks, $0.5N$ blocks are lost, could be solved by fixed sized partitions.

11. **Internal fragmentation**: use fixed-sized blocks and allocate memory in units based on block size - allocation partition may be much larger than requested memory, so unused space within the block is wasted. Still inefficient.

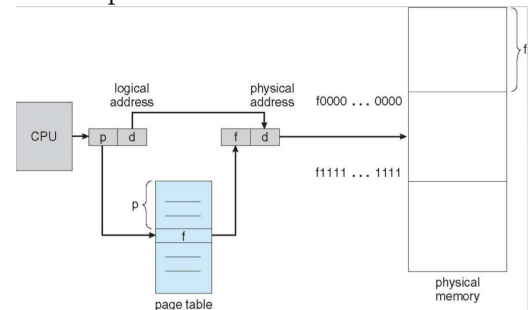
12. Fix fragmentation with: **Compaction**: shuffle memory to place all free memory in one contiguous block, but large overhead and not always possible; **Non-contiguous memory allocation**: allow process to be scattered throughout the memory - segmentation or paging.

13. **Segmentation**: divide program into dynamically-sized segments or contiguous memory blocks comprising $\langle \text{segment-number}, \text{offset} \rangle$. Use segment table to get $(\text{base} + \text{offset})$ to locate the segment in physical memory. Avoids internal but not external fragmentation.

14. **Segment table**: indexed by segment-number storing segment-base: starting physical address of segment memory block) and segment-limit, length of that segment.

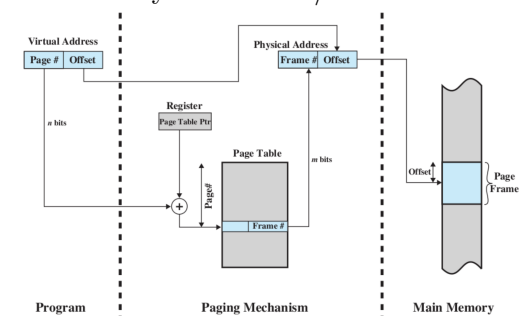
15. **Paging**: physical memory is divided into fixed-size blocks called **frames**, and a process's logical memory is split into **pages** of the same size (typically 4KB). Logical address $\langle \text{page-number}(p), \text{offset}(d) \rangle$ is translated using page table to physical address (frame-base+offset for MMU). For logical address of m bits with page size of 2^n bytes, lower n bits are the offset, upper $m - n$ are the page number. Pages are loaded into available frames for execution, preventing external fragmentation and the need for compaction, but can cause internal fragmentation of at most one page per process.

16. **Page table (PT)**: indexed by page-number from logical address, stores frame-number (frame-number $\times$ frame-size is starting physical addr of page's frame). Helps MMU translate virtual to physical addresses with the table stored in kernel space.



Memory 2

1. **Direct page table access**: need to store page base address in PCB and move to **page table base register (PTBR)** when process is scheduled. But ≥ 2 memory accesses to fetch every instruction/data in translation.



2. **Transition Lookaside Buffer (TLB)**: hardware cache to store up to 256 frequently used page table entries. When CPU generates logical address, page number checked against entries in parallel taking less than CPU cycle (near 0 overhead). Cache miss causes two memory accesses, missed entry brought into TLB by evicting existing one.

3. Cache placement algorithms include Least frequently used (LFU), least recently used (LRU) etc., each with some hit ratio upon the effective access time depends.

$$\text{Effective Access Time} = \text{Hit ratio} \times 1 \times \text{memory access} + (1 - \text{Hit ratio}) \times 2 \times \text{memory access}$$

4. **Address space identifier (ASID)** uniquely identifies process requesting the TLB search. Upon match, frame number returns, otherwise cache miss, (memory protection). Helps TLB support multiple processes.

5. Beneficial to have small page tables: modern systems use 32 or 64-bit logical addresses, e.g. 32-bit addr and 4KB page size, have $2^{32}/2^{12} = 2^{20}$ entries, if each requires 4 bytes, then page table size is 4MB per page table (process). Massive memory overhead, so only use a small number out of 2^{20} possible pages. **Valid-invalid bit** used for each page table entry to indicate if it has physical memory page corresponding to page number. But, wastes unused space, so combine into one entry.

6. **Hierarchical/Multi-level paging tables**: divide page table into pages, store each in a frame in memory. Outer page table stores that mapping, if it's bigger than one frame, can further divide it into pages. OS doesn't need to store inner PTs not in use.

7. **Hierarchical/Multi-level paging**: using same example, inner PTs would occupy $2^{10}=4\text{MB}/4\text{KB}$ frames. Outer PT will therefore have 2^{10} entries of 4 Bytes each, so total size is 4KB, fits into one frame, no further paging required. 10MS bits used to index into outer page, next 10 bits used to index into the inner page table, so 3 memory accesses instead of 2. Reduces memory overhead but increases number of memory accesses to read/write data upon TLB miss.

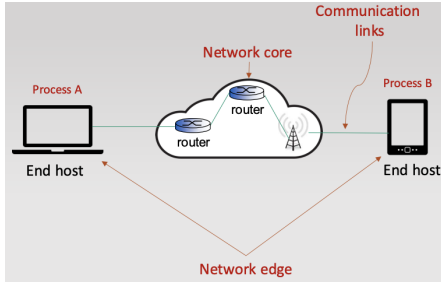
8. Page numbers are hash keys hashed to an index of PT, pointed to linked list of page numbers having same hash value. Each node contains page number, frame number, pointer to next node. Page numbers are compared in this chain searching for a match, if found - extract corresponding physical frame.

9. **Inverted Page Table**: each index corresponds to physical frame number. Entries comprise PID and page number, which are searched to be matched during virtual address generation. Decreases memory storing each page

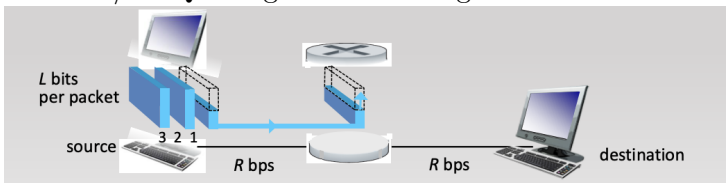
table, but increases time to search tables upon page reference.

Intro to Networks

1. **Network** is a system of inter-connected computing devices (end hosts), or pairwise network edges. Processes communicate through sending messages to each other.

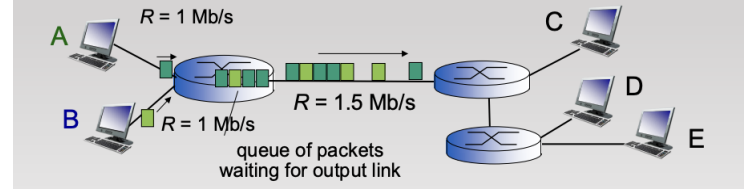


2. **Communication links** ($\leftrightarrow$) carry data between network devices as electromagnetic waves. Each link has a finite **bandwidth** (speed).
3. **Network edge** comprises **end hosts** which run network applications and communicate through the network core: $\text{Process}_A \leftrightarrow \text{network core} \leftrightarrow \text{Process}_B$
4. **Network core** comprises **packet switches** which help forward data packets using switches and routers.
5. **Routing**: runs routing algorithms to construct routing tables, **Forwarding** forwards arrived packets to the appropriate link according to routing table.
6. **IP address** identifies an end host uniquely within the network. **Port number** identifies a specific process running on that end host.
7. Each end-host must be able to:
 1. Run application processes generating messages.
 2. Break messages down into **packets** (smaller chunks).
 3. Add info (IP address, Port no) as **packet headers**.
 4. Send bits over a physical medium.
 Optionally: 5. provide reliable and orderly delivery of packets, and 6. control rate of transmission of packets.
8. **Store-and-forward** principle: entire packet must arrive at router before it can be transmitted to next link. Transmission (**push out**) of 1 L -bit packet into link at **transmission delay** R bps takes L/R seconds. End-to-end takes $2L/R$. Queuing when L is large.



9. **Queuing delay, loss**: if arrival rate to link exceeds transmission rate, then packets will queue, and may be dropped

(lost) if memory (buffer) fills up.



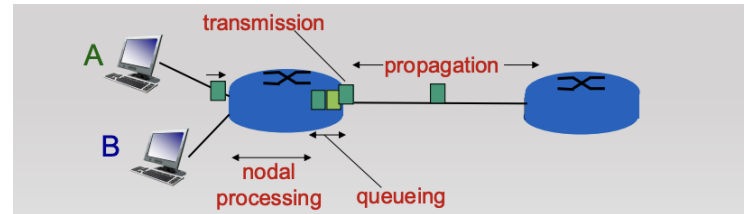
10. **Packet delay** $d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$.

Transmission delay $d_{\text{trans}} = L/R$ for packet length L (bits) and link bandwidth R (bits per second).

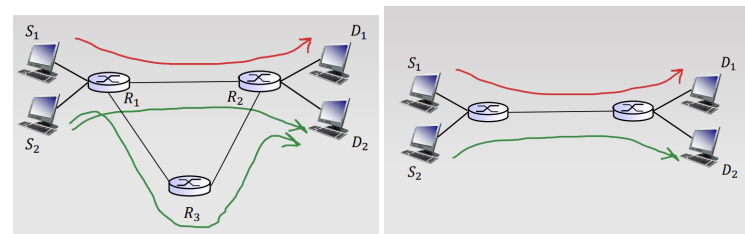
Queueing delay d_{queue} = time waiting at output link for transmission. Depends on congestion level of router.

Nodal processing d_{proc} : checks for bit errors and decides the output link at a network node, takes $< 1 \text{ ms}$.

Propagation delay $d_{\text{prop}} = d/s$ where d is physical link length, s is propagation speed in medium $3 \cdot 10^8 \text{ m/s}$



11. **Throughput** is specific to a **flow** of communicating pair. Rate of bit transfer from source to destination can be **instantaneous** $t = \frac{\Delta s}{\Delta t}$ or **average** $t = \frac{S}{T}$
12. **Bottleneck link** is one with minimal speed, limiting the overall throughput
13. **Protocol** is a set of rules for communicating nodes defining format and order of msgs exchanged and actions taken on them. Most are specified by **IETF** (Internet Engineering Task Force)
14. **Packet switching**: no pre-allocated resources (links) - different flows (src-dest pairs) share links along their routes. This traffic is **bursty**: if one flow isn't using a link, other flows can use it, or change routes if link fails (resources not pre allocated). **Adv**: well utilised resources, **Dis**: no rate guarantee, possible loss. (left fig.)

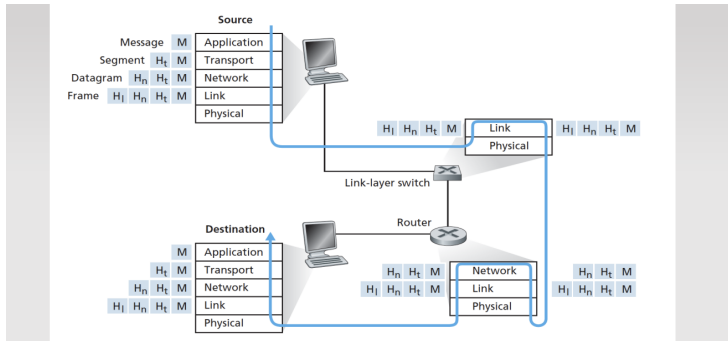


15. **Circuit switching**: circuit is reserved for each flow during entire call duration so can't use foreign circuits. Circuit comprises all links and nodes along a (src-dest) path. **Adv**: guaranteed rate, no loss, **Dis**: poor resource utilisation for bursty traffic. (*right fig.*)

16. **Layering**: layer N uses services of $N - 1$ and provides to $N + 1$. Easy to change layers without affecting others.

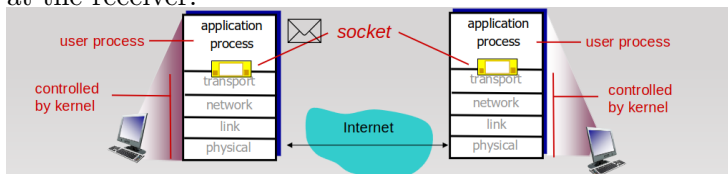
Application Layer Generate data to be communicated over the internet. HTTP SMTP DNS
Transport Layer Packetize the data, add port no., add sequencing and error correcting info TCP UDP
Network Layer Add source and destination IP addresses, Run routing algorithm IP Routing protocols
Link Layer Add source and destination MAC addresses, Pass frames onto NIC drivers Ethernet WiFi
Physical Layer Send individual bits through the physical communication link Separate protocol for each physical medium: co-axial cable, WiFi etc.

17. **Encapsulation**: each layer attaches its own header to the message before transmission. **Decapsulation**: at the receiver, each corresponding layer removes its own header to extract the useful data.



Application Layer 1 | TCP vs UDP, sockets

1. **Network application** is a system of processes sending messages over a network.
2. **Socket** is an API between application and transport layers, allowing to send/receive messages. 1. Application layer generates the message, 2. passes it through: transport, network and link layers, all adding their headers 3. physical layer transmits the packet, 4. packet is received at the receiver.



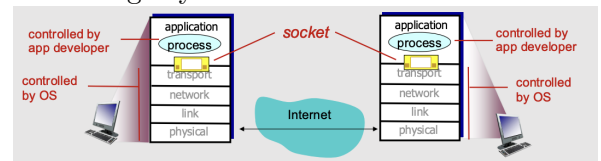
3. Messages to be addressed to correct process running within correct end host, so id by port numbers $\in [0, 2^{16} - 1]$, but $[0, 1023]$ reserved for well known network apps.

4. Application processes use transport layer services to deliver messages to intended recipients. App may require additional services from transport layer, e.g. in-order packet delivery. Can use TCP or UDP.

5. **Transmission Control Protocol (TCP)**: **reliable** and **in-order** data transfer service - packets can be lost or arrive out of order. Connection-oriented service, **TCP handshake**: connection established between client and server processes before they start communicating. Like phone call with checks, disconnect if hang up.

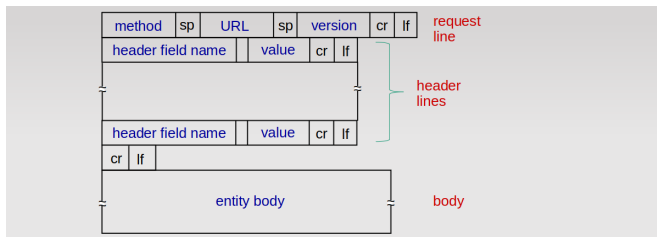
6. **User Datagram Protocol (UDP)**: **no guarantees** on data transfer, so no loss recovery (best effort service). Faster as no connection setup required, UDP headers smaller than TCP headers. Used in streaming. Like mail - no response speed guarantee and no notice before sending a message, can get lost or arrive out of order.

7. **Socket programming**: create connectionless (UDP) or connection oriented (TCP) sockets, read from and write to sockets through system calls.



Application Layer 2 | HTTPS v1.0 vs v1.1

1. Web browsers communicate with **web servers** using **HyperText Transfer Protocol (HTTP)** as application layer protocol (spec. by IETF). Uses TCP and port number 80. Client program does handshaking, then sends HTTP request, the server responds with HTTP message containing the requested web page and TCP closes.
2. Web page consists of objects (files) addressable by URL. **Non-persistent** (HTTP v1.0) uses separate TCP for each object, whereas **persistent** (HTTP v1.1) allows multiple objects per TCP connection.
3. **Non-persistent HTTP**: 1. client sends **SYN**, 2. server responds with **SYN-ACK**, 3. client sends **ACK** + object request, 4. server establishes connection and returns base HTML file, TCP connection closes. 5. Repeat for each referenced object. **RTT** is round trip time; response time for **each** object = $2\text{RTT} + \text{file transmission time}$, which is too slow.
4. In **persistent HTTP**, connection remains open after sending the response and subsequent HTTP requests are exchanged as soon as encountered, so $2\text{RTT} + \text{file transfer}$ in total (fast). HTTP request message general format:



HTTP request + response message example:

```
POST /api/login HTTP/1.1\r\n
Host: example.com\r\n
Content-Type: application/json\r\n
Content-Length: 45\r\n
\r\n
{"username":"alice","password":"secret"}
```

```
HTTP/1.1 200 OK\r\n
Date: Tue, 29 Apr 2025 12:34:56 GMT\r\n
Content-Type: application/json\r\n
Content-Length: 55\r\n
Connection: close\r\n
\r\n
{"status":"success","token":"eyJhbGciOiJI1NiIs..."}
```

5. **Web cache (proxy server)** sends popular data (if data in cache and up-to-date) without involving the origin server, otherwise requests it from origin server and forwards it to the user. Typically installed by Internet Service Provider (**ISP**) to serve requests of popular contents. Web caching reduces response time, and internet network traffic, hence cost.

Coursework | MAC, TCP headers, ARP, SYN

1. **Network interface** is the point of interconnection between device and network; a node can have multiple. Usually associated with a **hardware Network Interface Card (NIC)**. Each network interface has an IP address. Those associated with NIC have a MAC address.

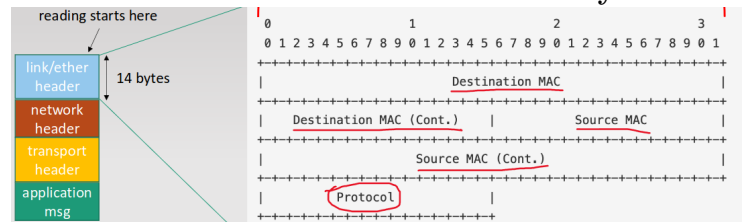
```
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.131.32.55 netmask 255.255.254.0 broadcast 10.131.33.255
    ether 74:86:e2:33:c2:a8 txqueuelen 1000 (Ethernet)
    RX packets 3474855 bytes 1950143051 (1.8 GiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 5466256 bytes 7488873422 (6.9 GiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 19 memory 0x72380000-723a0000
```

2. Localhost, or a **Loopback interface** is a **Virtual Network Interface**, so doesn't have a NIC or MAC.

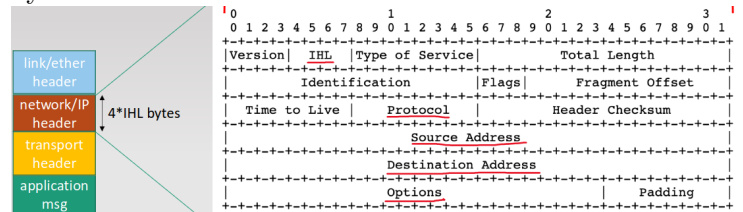
```
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 406 bytes 29032 (28.3 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 406 bytes 29032 (28.3 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

3. When application layer generates the message, the transport, network and link layers add their headers in that order. The message is then transmitted by the physical layer. To reconstruct the message, the recipient's layers strip corresponding headers in reverse order.

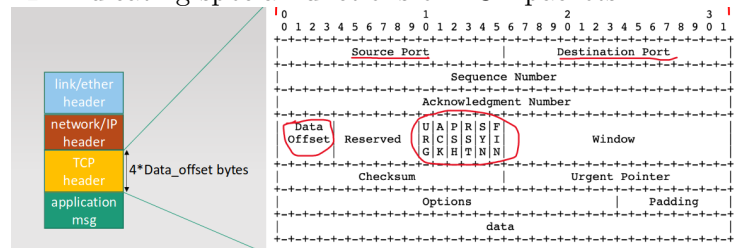
4. Link layer header (ethernet header) contains **6 byte** destination and **6 byte** source MAC, and a **2 byte** protocol of the next layer. Since the size is fixed, no padding is needed. Overall: 32 bits wide and stores **14 bytes**.



5. Network layer header (IP header) contains non-fixed Internet Header Length (**IHL**), taking **4 bits**, the **1 byte** protocol of the next layer and **32 bit** source and destination addresses. Overall: 32 bits wide and stores 4 * IHL bytes.



6. Transport layer header (TCP header) contains **16 bit** source and destination port numbers, identifying communicating processes. Also has a data offset taking **4 bits**, which describes the non-fixed length of the header in 4 byte words; **6 control bits**: URG, ACK, PSH, RST, SYN, FIN indicating special functions of TCP packets.



7. **SYN attack**: flood of SYN packets from spoofed src IPs create half-open connections, to which attacker doesn't send ACK - normal users connection denied: busy server.
8. **MAC** address uniquely identifies network interfaces.
9. **Address Resolution Protocol (ARP)** maps a known IP address to an unknown MAC address on the same local network (link). ARP works by broadcasting a request "Who has IP ...? Tell me your MAC. My MAC is ..." to all devices on the link. The device with that IP replies with its MAC address. The sender stores this IP-to-MAC mapping in its ARP cache for future use.

10. **ARP cache poisoning** attack: ARP allows unsolicited replies since IP can change, so an attacker can pretend to be someone else, hence receiving the messages meant for somebody else.

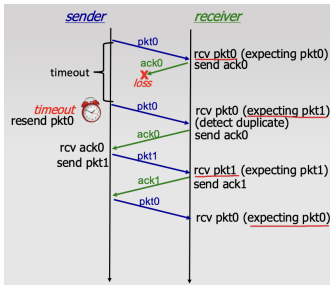
1. Transport layer provides logical communication between app processes running on different hosts. Has **send side**: breaks msgs into **segments**, adds header and passes to network layer, and **receive side**: reassembles segments into msgs, passes to app layer.

2. **UDP** or **User Datagram Protocol** has no packet recovery, is **connectionless** - each segmented is treated independently, **no congestion control**: any send rate is allowed even when network is congested. No connection establishment, smaller header, hence faster so used for video streaming and telephony.

3. **TCP or Transmission Control Protocol** extends UDP with **reliable data transfer** (recover loss, packets arrive in order), **flow control**: matches sending speed to receiver reading speed and **congestion control**: send rate depends on perceived network congestion.

4. Problem: unreliable channel (network layer) may contain **bit errors**: bits flipped due to electrical noise, **packets loss and reordering** due to buffer overflow at routers.

5. **Reliable transfer** uses **checksums** (include sum of all 16-bit words in the header) to detect bit errors. **ACKs** to indicate if packet is correctly received at receiver, else use **timeout mechanism**. **Retransmissions** re-send corrupted packets using **Automatic Repeat Request** (ARQ). Finally use **sequence number** to order packages.



6.

7. Link capacity is under-utilised with Stop and Wait ARQ, given $R = 10^9 bps$ link, packet length $L = 8000$ bits, RTT of $30ms$ - get extremely low utilisation U .

$$U = \frac{L}{R \times \text{RTT} + L} = 0.00027$$

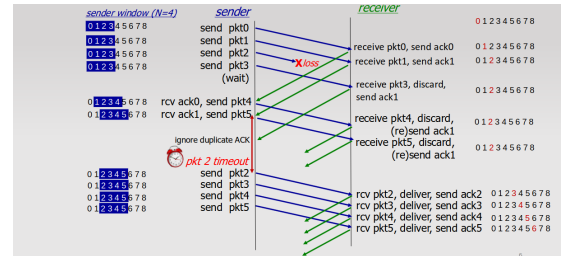
8. Instead, allow the sender to send more packets through a **pipeline** without waiting for ACK. Pipeline can handle $R \times \text{RTT}$ bits in that time (**delay bandwidth product**).

9. Using a pipeline and the original packet of length L , can send at most $(L + R \times \text{RTT} \simeq R \times \text{RTT})$ bits: L is negligible.

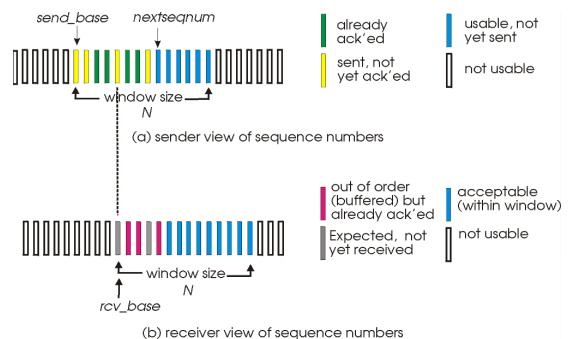
10. Receiving process has a finite **buffer** of B bits, so sender should send at most $\min(B, L + R \times RTT)$ bits without waiting for ACK.

1. **Pipelined protocols** allow multiple unacknowledged packets in the pipeline, ACK is sent individually or cumulatively, range of sequence numbers is increased.

2. **Go-Back-N**: can send up to N packets without waiting for ACK, receiver maintains `expectedseqnum` of the next expected sequence number to be received (to maintain packet order). If packet n is correctly received and `expectedseqnum=n`, then sends **cumulative** `ACK(n)` to all packets up to and including n and increments `expectedseqnum`. Else ($\neq n$) discard incoming packet and send `ACK(expectedseqnum-1)` to indicate the last correctly recieved packet. **GBN** in action:



3. **Selective Repeat (SR)** Accepts packets as long as they're within the **receive window**; ACKs are individual. The sender selectively retransmits packets whose ACKs didn't arrive (based on per-packet timers in the send window). The sender moves its send window upon receiving ACKs; the receiver moves its receive window upon delivering in-order packets to the application. Out-of-order packets are buffered, so there's no need to retransmit them.



4. **TCP reliable data transfer (RDT)** combines the two: uses cumulative ACKs and only retransmits segment causing timeout. In TCP, each byte is numbered. **Sequence number** is byte number of first byte of the TCP segment.

5. **TCP acknowledgement number** is the number of the next expected byte from the other side, is cumulative.

6. **TCP duplex communication:** in TCP, data can flow in both directions, so a segment can both carry data and serve as an ACK.

- TCP timeout period is long, so use TCP **fast retransmit**: upon receiving 3 duplicate ACKs for a segment, TCP sender retransmits that segment without waiting for timeout, as duplicate ACKs signify packet loss.
- To summarise, TCP header also contains a 32-bit **sequence** ("have") number (first byte), and an **acknowledgement** ("need") number of the next byte expected.

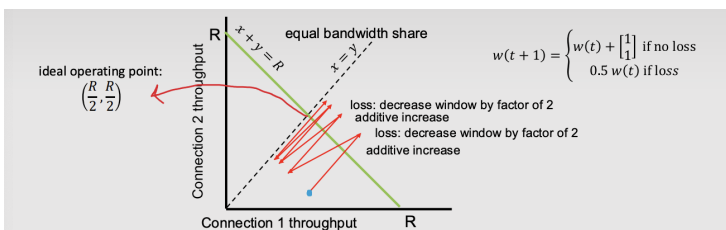
Transport Layer 3 | flow & congestion control

- Flow control**: data pipeline has limited receive buffer size, so sending speed must not exceed the "advertised" free buffer space within the **receive window field** (rwnd).

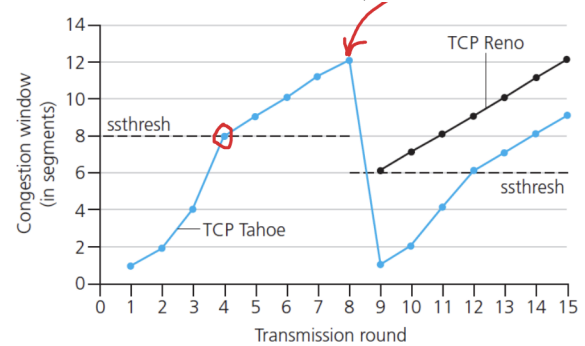
$$W = \text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$$
Sender limits amount of unacked ("in-flight") data to receiver's rwnd, so no buffer overflow.
- Congestion control**: set transmission rate according to the **perceived** congestion s.t. each flow gets a "fair" share of the network resources. Congestion at router is when $\text{in_rate} > \text{out_rate}$ - may cause lost packets (buffer overflow) and long delays (buffer queueing).
- Given transmission rate λ of each of n senders and throughput λ_{out} of each receiver: The delay goes to ∞ when $\lim_{\text{link} \rightarrow \frac{C}{n}}$ for transmission capacity C .

- Need to avoid network congestion and provide a fair share of network resources to each flow, but no need to increase transmission rate beyond the network capacity. TCP takes timeout more seriously than 3 duplicate ACKs.
- Congestion window** cwnd limits # unacked data "in-flight"

$$W = \text{LastByteSent} - \text{LastByteAcked} \leq \min(\text{cwnd}, \text{rwnd})$$
Controlling W controls transmission rate $\approx W/\text{RTT}$ Bps. With **Maximum Segment (packet) Size (MSS)**, the number of segments in flight is $\lfloor W/\text{MSS} \rfloor$. When rwnd is large: rate $\approx \text{cwnd}/\text{RTT}$; cwnd adjusts dynamically to perceived congestion.
- Additive increase multiplicative decrease (AIMD)**: each RTT **additively increase** transmission rate (cwnd) by 1 MSS until loss occurs (cwnd+1MSS), if it does - **multiplicatively decrease** by factor of 2 (cwnd÷2). AIMD is fair, if 2 TCP senders share same bottleneck link of capacity R , then each will have throughput of $\frac{R}{2}$.



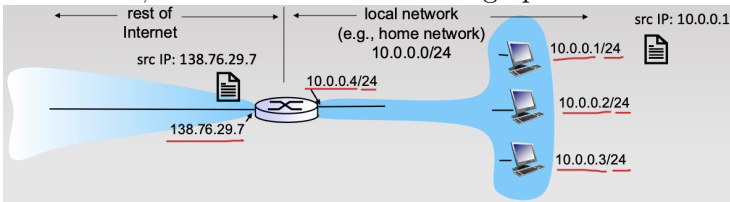
- However, convergence for AIMD is slow (linear), so use **slow start phase**, where cwnd is increased exponentially starting from small value (1MSS) until **slow start threshold ssthresh** is reached (start AIMD), or loss is detected (re-enter slow start). **ssthresh** remembers the previous window size for which a loss occurred, so self-adjusts in the two following ways: Duplicate ACKs, Timeout.
- Duplicate ACKs**: some segments still received, so be lenient, set $\text{ssthresh} = \text{cwnd}/2$ cut cwnd in half: $\text{cwnd} = \text{cwnd}/2$ then grow linearly $\pm 1\text{MSS}$ (soft reset).
- Timeout** loss needs drastic action: set $\text{ssthresh} = \text{cwnd}/2$ and hard-reset $\text{cwnd} = 1\text{MSS}$, re-enter **slow start** phase.



Network Layer

- Network layer** moves packets from src to dest through intermediate nodes (**routers**). Uses **Internet Protocol (IP)** throughout: *at source*, the IP header containing src, dest IP addresses is added and sent to link layer. *At routers*: routes packets depending on dest IP. *At destination*: receive IP datagram, strip IP header, deliver to transport layer. Uses routing protocols to compute routes.
- Forwarding**: move packets from router's input to appropriate router output. **Routing**: constructing **routing table** given some protocol. Too many (2^{32}) possible IP's, so use groups/ranges of IPs as keys in the lookup table mapping to appropriate interfaces use groups of IPs: e.g. 0..000 through 0..100 map to link interface 1 etc.
- Longest prefix matching**: in forwarding table, use longest address prefix matching destination address to reduce the route lookup time (direct to most relevant addr).
 110010 vs 110100 - $5 > 3$, choose left
- IPv4** addresses uniquely identify network interfaces with 32-bits (4 bytes) in dotted decimal:
 $223.1.1.1 = 11011111 \ 00000001 \ 00000001 \ 00000001$
- IP addresses within the same **subnet** share the same prefix or **subnet mask**, connected by link layer switch and can communicate with each other directly through MAC.

6. **Classless Inter-Domain Routing (CIDR)** of form $a.b.c.d/x$ is the IPv4 address with a subnet mask $/x$ - x bits from the left specify the shared network, the other $32-x$ bits specify hosts within that subnet.
7. **Default gateway** (router) connects subnets. If src and $dest$ belong to different subnets - forward the packet to src 's default gateway. e.g. A sends message m to B : m is forwarded to A 's default gateway R (router) that looks up the routing table and forwards m to an appropriate outgoing interface towards B . From there, m is forwarded to B through the switch in B 's subnet.
8. Nodes acquire IP address either through manual admin configuration (e.g. `/etc/rc.config`) or **Dynamic Host Configuration Protocol (DHCP)** - application layer protocol that dynamically assigns IP address from a server (usually gateway router) to clients (nodes trying to connect to subnet). Both methods require subnet mask and default gateway specification.
9. Networks acquire subnet part of IP address from internet services provider's (ISP) address space, who in turn get their 2^{12} IP addresses from ICANN global authority.
10. Not enough IPv4 addresses - impossible for ISPs to provide a unique one to each device connected to a subnet - instead assign unique **public IP addresses** to gateway routers. Assign **private IP addresses** to other devices, unique in subnet but not globally.
11. Private IP addresses can communicate with each other on same subnet, but can't be carried through public internet.



Network address translation (NAT) converts private source IP address of private hosts into public IP address of their gateways. Host with address A sends message m to external server with address B through a NAT router, which records $A \leftrightarrow B$ mapping and uses the public address to send the message. When received - use this mapping to decode the recipient. If more than 1 recipient - open multiple ports. But better use IPv6.

12. **Routing protocol** constructs the routing table at each gateway (router), using some algorithm.
13. **Network Graph** $G = (N, E)$ has a set N of routers and set E of links between them. Each edge $e = (x, y) \in E$ has a cost $c(x, y)$, $c(x, y) = \infty$ if x, y aren't neighbours.

14. Routing algorithms can be **global**, requiring knowledge of complete topology at each router, including costs (**link state algorithms**), or **local**: only requiring knowledge of local neighbourhood at each router.
15. **Link-State routing** algorithm: Dijkstra's, computes least cost from source node to all others. Implemented in Open Shortest Path First (OSPF) protocol. Obtains topology through broadcasting of link costs (states) - each node "advertises" cost of all the links connected to it.
16. **Bellman-Ford equation**: given length of shortest path $d_x(y)$ from x to y and neighbours $N(x)$ of x :

$$d_x(y) = \min_{v \in N(x)} \{c(x, v) + d_v(y)\}$$

Hop to router $v^* \in N(x)$ if v^* minimises the above sum.

17. **Distance Vector (DV) algorithm** is implemented in **Routing Information Protocol (RIP)** - uses local information from neighbouring nodes to find shortest paths using Bellman-Ford Equation.
 18. **DV algorithm**: $D_x(y) \neq d_x(y)$ is the current estimate of minimum distance from x to y . Try to converge these estimates to actual values. Each node x maintains distance vector $D_x = [D_x(y) : y \in N]$ of current estimates, and node x performs update $D_x(y) = \min_v \{c(x, v) + D_v(y)\}$, where the distance vector of each neighbour v , D_v is obtained through message passing. After every update - sends the updated distance vector to all its neighbours.
- Each node: 1. wait for change in local link cost or message from neighbour, 2. recompute estimates using Bellman-Ford equation, 3. if DV to any destination has changed - notify neighbours.

end.

Metric	Formula / Description
Propagation delay	$d_{\text{prop}} = \frac{\text{distance}}{\text{signal speed}}$. Example: 90 km in fiber (3×10^8 m/s) $\simeq$ 0.3 ms.
Transmission delay	$d_{\text{trans}} = \frac{L}{R}$, where L = packet length (bits), R = link bandwidth (bps). Example: 1500-byte (12,000-bit) packet on 100 Mbps link = 0.12 ms.
Queuing delay	No simple formula (depends on traffic). Grows as traffic approaches link capacity. Often modeled probabilistically (arrival rate λ , service rate μ).
Round Trip Time (RTT)	Time for a small packet to go from client to server and back. Roughly $\text{RTT} \approx 2 \times d_{\text{prop}}$ (if processing and transmission are negligible). Measures network latency.
Throughput	Rate of successful data delivery (bits/sec). Limited by the slowest link or sender/receiver capability. Example: links of 10, 5, 8 Mbps $\rightarrow$ max throughput = 5 Mbps (bottleneck).

Table 1: Important Network Metrics and Their Formulas

Aspect	TCP	UDP
Connection	Connection-oriented (handshake)	Connectionless (no handshake)
Reliability	Reliable delivery with ACKs and retransmissions	Unreliable, no guaranteed delivery
Ordering	In-order delivery guaranteed	No ordering guarantee
Congestion Control	Built-in congestion control	No congestion control
Flow Control	Uses flow control (receiver window)	No flow control
Header Size	Larger header (20 bytes)	Smaller header (8 bytes)
Transmission Style	Stream-oriented (continuous byte stream)	Message-oriented (datagrams)
Setup or Teardown	3-way handshake setup and teardown	No setup or teardown
Use Cases	Web, file transfer, email (reliable apps)	Streaming, DNS (speed-critical)
Broadcast/Multicast	No support	Supports multicast/broadcast
Analogy	Email (reliable conversation)	Phone call (noise, lossy)

Table 2: Comparison of TCP and UDP

Scenario	What's Lost?	TCP Response	Key Concept
Lost ACK	ACK	Retransmit data	ACK loss triggers timeout
Delayed ACK	ACK (delayed)	Retransmit prematurely	Delayed ACK causes unnecessary retransm.
Cumulative ACK	ACK	No duplicate data sent	ACK confirms multiple segments
Lost Packet	Data segment	Retransmit missing segment	Duplicate ACKs and buffering

Table 3: Comparison of TCP retransmission scenarios